

Projeto Final em Engenharia Informática

Espetrograma

ANÁLISE E INTERPRETAÇÃO DE ÁUDIO PARA UTILIZADORES NÃO
ESPECIALISTAS

RELATÓRIO FINAL

Paulo Silva - 2100537

Pedro Pestana

24-06-2026

Índice

Organização do Relatório	6
Capítulo 1 – Introdução.....	7
Descrição e objetivos do trabalho	7
Proposta para o trabalho	7
Objetivos	7
Resultados esperados.....	8
Restrições	9
Restrições de tempo.....	9
Restrições técnicas	9
Restrições arquiteturais	9
Restrições de âmbito académico	10
Planeamento Geral.....	10
Capítulo 2 – Desenho.....	12
Fundamentos/Estado-da-arte.....	12
2.1. Arquitetura adotada.....	12
2.2. Fundamentos de processamento de sinal de áudio	13
2.2.1 Transformada de Fourier de Curto Prazo (STFT)	13
2.2.2 Detecção de Silêncio	14
2.2.3 Detecção de Clipping	15
2.2.4 Análise de Ruído de Fundo	15
2.2.5 Detecção de Eventos Sonoros	16
2.3 Decisões de engenharia e alternativas consideradas	17
2.4. Tecnologias principais	18
2.4.1 Backend	18
2.4.2 Frontend	19
2.4.3 Testes	20
2.5 Requisitos e Priorização (MoSCoW)	20
2.5.1 Requisitos Must Have	21
2.5.2 Requisitos Should Have.....	21
2.5.3 Requisitos Could Have	22
Capítulo 3 – Implementação.....	23

3.1. Estrutura do protótipo	23
3.2. Funcionalidades implementadas	24
3.2.1 Upload de Áudio (Mo1) — CONCLUÍDO	24
3.2.2 Geração e Visualização do Espectrograma 2D (Mo2) CONCLUÍDO	25
3.2.3 Detecção de silêncio (Mo3) — CONCLUÍDO	26
3.2.4 Detecção de Clipping (Mo4) — CONCLUÍDO	26
3.2.5 Análise de Ruído de Fundo (Mo5) — CONCLUÍDO	27
3.2.6 Detecção de eventos sonoros relevantes (Mo6) — CONCLUÍDO	27
3.2.7 Anotação Visual de Eventos (Mo7) — CONCLUÍDO	28
3.2.8 Feedback Automático em Linguagem Simples (Mo8) — CONCLUÍDO	28
3.2.9 Interface para Não Especialistas (Mo9) — CONCLUÍDO	29
3.2.10 Vista 3D do Espectrograma (Co1) — CONCLUÍDO [EXTRA]	34
3.3 Estado de implementação do MVP	34
3.4 Evidências de execução e validação	35
3.4.1 Testes automatizados	35
3.4.2 Validações funcionais	37
3.5 Decisões técnicas relevantes	38
Capítulo 4 – Testes	40
4.1 Estratégia geral de testes	40
4.2 Testes automatizados	40
4.2.1 Testes de análise de áudio	41
4.2.2 Testes de feedback	42
4.2.3 Ambiente de testes e fixtures	43
4.3 Validações funcionais	43
4.4 Cobertura face à especificação	44
Capítulo 5 – Conclusões	46
5.1 Resultados face aos objetivos	46
5.2 Dificuldades e limitações	47
5.3 Possíveis melhorias	48
5.4 Trabalho futuro	49
Anexo I	50
Bibliografia	51

Lista de Figuras

Figura 1- Visão geral da arquitetura	23
Figura 2 - Página de upload	30
Figura 3 - Página de análise 2D.....	31
Figura 4 - Vista 3D	32
Figura 5 - Página de feedback.....	33

Introdução

O projeto Espectrograma surgiu da necessidade de tornar a análise de áudio acessível a utilizadores sem formação técnica. As ferramentas profissionais existentes — como Audacity, Adobe Audition ou iZotope RX — exigem familiaridade com conceitos de processamento de sinal (por exemplo, STFT, RMS ou centroide espectral) para que os resultados apresentados possam ser corretamente interpretados. Este fosso cria uma barreira para estudantes, jornalistas, podcasters e criadores de conteúdo que apenas pretendem responder a perguntas simples, como: “O meu áudio tem muita distorção?”, “Há muito ruído de fundo?” ou “Onde estão as pausas?”.

Para responder a esta necessidade, o projeto propõe o desenvolvimento de uma aplicação web centrada no espectrograma como representação visual do som. A aplicação recebe um ficheiro de áudio, aplica um pipeline de análise de sinal em Python e apresenta o resultado sob a forma de espectrograma enriquecido com anotações visuais e descrições em linguagem simples. Desta forma, métricas técnicas são traduzidas em indicações intuitivas sobre a qualidade do áudio, permitindo que utilizadores não especialistas compreendam rapidamente onde existem problemas como ruído de fundo, distorção ou silêncios prolongados.

Na fase final do projeto, o MVP (Minimum Viable Product) encontra-se completo: todos os nove requisitos Must Have definidos na especificação foram implementados e validados. Para além deste núcleo funcional, foi ainda desenvolvida uma visualização 3D do espectrograma (requisito Could Have Co1), recorrendo a Three.js e WebGL, bem como um conjunto de otimizações de desempenho, incluindo a renderização 2D em Web Worker e o uso de cache em sessionStorage para reduzir tempos de carregamento e melhorar a experiência de utilização.

A partir deste enquadramento geral, nas secções seguintes da Introdução serão detalhados o âmbito do projeto, os objetivos definidos, as restrições consideradas, os resultados esperados e o planeamento global do trabalho.

Organização do Relatório

O relatório está organizado em cinco capítulos:

- **Capítulo 1 – Introdução:** *apresenta o enquadramento do projeto, a motivação, os objetivos, as restrições, os resultados esperados e o planeamento geral.*
- **Capítulo 2 – Desenho:** *descreve as tecnologias selecionadas, a arquitetura do sistema, o modelo de dados, os principais algoritmos e a interface do utilizador, justificando as opções tomadas.*
- **Capítulo 3 – Implementação:** *detalha a implementação do protótipo, a decomposição do trabalho em tarefas, a calendarização e o estado final de cada componente.*
- **Capítulo 4 – Testes:** *apresenta os testes realizados, incluindo exemplos de funcionamento, testes de funcionalidade e desempenho, bem como os resultados obtidos.*
- **Capítulo 5 – Conclusões:** *discute os resultados face aos objetivos, identifica limitações, possíveis melhorias e trabalho futuro.*

Capítulo 1 – Introdução

Descrição e objetivos do trabalho

Este capítulo contextualiza o problema abordado, define o âmbito do projeto e explicita os objetivos e resultados esperados que orientam o desenvolvimento da aplicação Espectrograma.

Proposta para o trabalho

O projeto Espectrograma consiste no desenvolvimento de uma aplicação web para análise de ficheiros de áudio nos formatos WAV e MP3, com geração de espectrograma 2D e deteção automática de eventos relevantes no sinal. O sistema recebe o ficheiro de áudio, processa-o com recurso a bibliotecas científicas e apresenta os resultados através de elementos visuais e textuais compreensíveis para utilizadores não especialistas.

O âmbito da solução centra-se na implementação de um MVP (Minimum Viable Product) com processamento local no backend e uma interface orientada a um fluxo simples de utilização:

upload → análise → interpretação. Nesta fase, não são considerados componentes como base de dados persistente, autenticação de utilizadores ou processamento em tempo real, de forma a manter o foco na robustez e clareza do pipeline principal de análise de áudio e visualização do espectrograma.

Objetivos

Os objetivos gerais do projeto são:

- Tornar a análise básica de qualidade de áudio acessível a utilizadores sem formação técnica em processamento de sinal.
- Explorar o espectrograma como representação visual central, enriquecida com anotações que facilitem a interpretação de eventos sonoros relevantes.

A partir destes objetivos gerais, foram definidos os seguintes objetivos funcionais para o MVP:

1. Permitir o upload validado de ficheiros de áudio (WAV/MP3) até 10 MB.
2. Gerar e visualizar um espectrograma 2D com eixos temporal e de frequência.
3. Detetar segmentos de silêncio e apresentar os respetivos timestamps.
4. Detetar clipping e sinalizar picos de saturação no sinal.
5. Classificar o ruído de fundo em níveis qualitativos (baixo, moderado, alto ou desconhecido quando o sinal não apresenta variação dinâmica suficiente).
6. Detetar eventos sonoros com base em variações de energia, variação espectral e transientes.
7. Anotar visualmente diferentes tipos de eventos diretamente sobre o espectrograma.
8. Produzir feedback textual em linguagem simples, traduzindo métricas técnicas em descrições intuitivas.
9. Disponibilizar uma interface utilizável por não especialistas, com um fluxo de interação simples e direto

Resultados esperados

Como resultado da execução do projeto, espera-se obter:

- Um protótipo funcional de aplicação web que implemente todos os objetivos funcionais definidos para o MVP.
- Um pipeline de análise de áudio robusto, capaz de processar ficheiros WAV/MP3 até 10 MB e gerar espectrogramas 2D com anotações visuais de eventos relevantes.
- Um conjunto de métricas e heurísticas que permitam identificar silêncio, clipping, ruído de fundo e eventos sonoros de forma consistente com o âmbito do projeto.
- Uma interface gráfica que permita a utilizadores não especialistas carregar ficheiros, visualizar o espectrograma e interpretar os resultados através de anotações visuais e feedback textual.
- Documentação técnica e de utilização que descreva a arquitetura da solução, as principais decisões de desenho e as limitações identificadas, servindo de base a

evoluções futuras (por exemplo, visualização 3D, persistência de dados ou processamento em tempo quase real).

Restrições

O desenvolvimento foi condicionado por restrições de quatro naturezas.

Restrições de tempo

O projeto decorre num único semestre letivo (17 de março a 24 de junho de 2026), totalizando 16 semanas. Este horizonte impôs a necessidade de um âmbito bem delimitado e de priorização estrita dos requisitos, com recurso ao método MoSCoW para separar o que é essencial do que é desejável.

Restrições técnicas

Formatos de áudio limitados a WAV e MP3, que cobrem os casos de uso mais comuns para o público-alvo. Formatos sem perdas (FLAC, AIFF, OGG) foram explicitamente excluídos do âmbito.

Tamanho máximo de ficheiro fixado em 10 MB, correspondendo a aproximadamente 5 minutos de MP3 a 128 kbps ou 2 minutos de WAV a 44 100 Hz / 16 bits. Este limite é imposto pelo servidor Flask em modo single-threaded, que processa um pedido de cada vez — ficheiros maiores tornariam o tempo de resposta inaceitável.

Processamento exclusivamente local no servidor: não são utilizados serviços externos de terceiros (APIs de transcrição, modelos de ML hospedados na cloud). Toda a análise é realizada com bibliotecas Python instaladas localmente (librosa, NumPy), garantindo privacidade dos dados e independência de conectividade.

Restrições arquiteturais

Sem processamento em tempo real: o sistema analisa ficheiros pré-gravados enviados por upload. A arquitetura de resposta síncrona HTTP não suporta streaming de áudio ao vivo, que exigiria WebSocket ou WebRTC e uma pipeline de baixa latência.

Sem persistência entre sessões: não existe base de dados. O estado é mantido em Flask Session (cookie assinado) durante uma sessão de browser e descartado ao fechar ou ao acionar /reset. Não há histórico de análises anteriores.

Sem autenticação de utilizadores: a aplicação é de acesso aberto, sem contas ou controlo de acesso. Qualquer utilizador com acesso ao servidor pode fazer upload e analisar ficheiros.

Restrições de âmbito académico

O projeto é desenvolvido individualmente, no contexto da unidade curricular Projeto Final em Engenharia Informática da Universidade Aberta. O âmbito é definido de forma a ser exequível por um único estudante em regime pós-laboral, sem equipa de suporte ou recursos de infraestrutura dedicados.

Planeamento Geral

O trabalho foi organizado em fases, alinhadas com os marcos de avaliação da unidade curricular. De forma simplificada, o planeamento geral pode ser descrito da seguinte forma:

Fase 0 – Identificação e escolha do tema

Leitura do enunciado da unidade curricular e análise das propostas de projeto disponíveis.

Escolha do tema Espectrograma e contacto inicial com o orientador.

Fase 1 – Especificação inicial

Definição do problema, âmbito e objetivos do projeto.

Identificação dos requisitos funcionais do MVP (lista de Must Have).

Entrega da proposta inicial com aceitação do orientador.

Fase 2 – Desenho e prototipagem

Escolha das tecnologias a utilizar no backend e frontend.

Desenho da arquitetura da aplicação e do pipeline de análise de áudio.

Implementação de um protótipo inicial com upload de ficheiros e geração básica de espectrograma 2D.

Fase 3 – Implementação do MVP

Implementação das funcionalidades de deteção de silêncio, clipping, ruído de fundo e eventos sonoros.

Desenvolvimento das anotações visuais sobre o espectrograma e do feedback textual em linguagem simples.

Otimizações de desempenho e melhorias de usabilidade na interface web.

Fase 4 – Testes e validação

Definição de um conjunto de ficheiros de áudio de teste com diferentes características (níveis de ruído, distorção, pausas, etc.).

Testes funcionais às principais funcionalidades do MVP.

Ajustes finos aos limiares e heurísticas de deteção, com base nos resultados obtidos.

Fase 5 – Conclusões e trabalho futuro

Análise crítica dos resultados face aos objetivos definidos.

Identificação de limitações da solução atual.

Proposta de evoluções futuras, como visualização 3D, persistência de dados ou suporte a novos formatos.

Este planeamento serve de guia para o desenvolvimento do projeto e será detalhado e atualizado nos capítulos seguintes, em particular no que respeita à implementação e aos testes realizados.

Capítulo 2 – Desenho

Fundamentos/Estado-da-arte

Este capítulo apresenta os fundamentos tecnológicos que suportam o protótipo Espectrograma e descreve as principais decisões de engenharia adotadas. São detalhados a arquitetura da aplicação, os conceitos de processamento de sinal de áudio utilizados e os algoritmos implementados para detecção de silêncio, clipping, ruído de fundo e eventos sonoros. O objetivo é justificar as opções tomadas, evidenciando o equilíbrio entre simplicidade, prazo de desenvolvimento e capacidade de evolução futura.

2.1. Arquitetura adotada

Foi adotada uma arquitetura web em três camadas, com separação clara de responsabilidades:

- **Frontend**
 - Interface HTML/CSS/JavaScript, responsável pelo upload do ficheiro, visualização interativa do espectrograma e apresentação do feedback ao utilizador. Esta camada não executa processamento pesado de áudio, focando-se na experiência de utilização e na representação gráfica dos resultados.
- **Backend (API Flask)**
 - Serviço HTTP que recebe o ficheiro por multipart/form-data, valida-o, invoca o módulo de processamento e devolve os resultados serializados em JSON. O estado mínimo da sessão (por exemplo, nome do ficheiro e sumário de análise) é mantido em Flask Session, através de um cookie assinado com SECRET_KEY.
 - Módulo de Processamento (Python)

Componente responsável pelo pipeline de análise de sinal, executado de forma síncrona dentro do processo Flask. Não existe base de dados persistente, todos os dados são gerados em memória e descartados após o envio da resposta HTTP. Esta separação facilita a manutenção, os testes por componente e a evolução incremental sem acoplamento excessivo entre camadas.

No futuro, o módulo de processamento poderá ser isolado num serviço independente ou escalado separadamente, sem alterações significativas ao frontend.

2.2. Fundamentos de processamento de sinal de áudio

2.2.1 Transformada de Fourier de Curto Prazo (STFT)

Um ficheiro de áudio digital é uma sequência de amostras — valores de amplitude medidos a uma taxa fixa (taxa de amostragem, tipicamente 44 100 Hz para áudio de qualidade CD). Uma transformada de Fourier aplicada ao sinal completo produziria um espectro médio que oculta a evolução temporal das frequências.

A Transformada de Fourier de Curto Prazo (STFT — *Short-Time Fourier Transform*) resolve este problema ao aplicar a Transformada Discreta de Fourier (DFT) sobre janelas sobrepostas do sinal. Para cada janela de N amostras (`n_fft`), a DFT decompõe o sinal nas suas componentes de frequência. O resultado é uma matriz complexa $D[f, t]$ com dimensões frequência $\times$ tempo.

O espectrograma de potência obtém-se por $|D[f, t]|^2$, convertido para a escala logarítmica em decibéis (dB) com a fórmula:

$$S_{dB}[f, t] = 10 \times \log_{10} \left(\frac{|D[f, t]|^2}{\text{ref}} \right)$$

onde *ref* é o valor máximo de potência (normalização relativa ao pico).

Na implementação (**audio_analysis.py**), são usados os seguintes parâmetros:

- **n_fft = 4096** → resolução de frequência: $sr/n_fft \approx 10,7$ Hz por *bin* (para **sr = 44100** Hz)
- **hop_length = 256** → passo temporal: $256/44100 \approx 5,8$ ms por coluna

Janela de análise: **Hann** (padrão do **librosa**)

A matriz resultante é normalizada para o intervalo [0, 255] (**uint8**) para transmissão eficiente ao frontend como *array* JSON, onde é posteriormente mapeada para uma paleta de cores.

2.2.2 Detecção de Silêncio

A detecção de silêncio baseia-se no perfil de energia RMS calculado com `frame_length = 2048` e `hop_length = 256`. Em vez de um limiar fixo em dB, foi adotada uma abordagem adaptativa que determina o limiar de silêncio em função da distribuição energética do próprio sinal:

$$silence_thresh = P_{10}(RMS) + (P_{90}(RMS) - P_{10}(RMS)) \times 0.15$$

Esta formulação posiciona o limiar a 15% do intervalo dinâmico do sinal, entre o piso de ruído (percentil 10) e o nível típico de sinal ativo (percentil 90). A vantagem face a um limiar fixo em dB é a independência do ganho da gravação: ficheiros muito silenciosos e ficheiros muito ruidosos são tratados com a mesma robustez, sem necessidade de ajuste manual.

Um frame é classificado como silencioso quando a sua energia RMS é inferior ao limiar calculado. Os intervalos de silêncio são obtidos como as regiões contíguas de frames silenciosos, convertidos para timestamps (início/fim em segundos). São descartados os segmentos com duração inferior a 120 ms, evitando falsos positivos causados por microinterrupções naturais na fala.

2.2.3 Detecção de Clipping

O **clipping** (saturação) ocorre quando a amplitude do sinal ultrapassa o limite de representação do sistema de aquisição. Em áudio digital de 16 bits, isso corresponde a amostras próximas de ± 32767 após normalização para o intervalo $[-1.0, 1.0]$ do **librosa**. O limiar é $|y[n]| \geq 0.99$. Para evitar falsos positivos em sinais altos mas limpos, exige-se ainda um mínimo de **3 amostras consecutivas** acima do limiar — condição de flat-top que distingue saturação real de amplitude elevada.

O algoritmo identifica todas as amostras acima do limiar, agrupa-as em segmentos contínuos e funde segmentos com separação inferior a 50 ms. Esta fusão evita a fragmentação visual causada por clipping intermitente de alta frequência, resultando em anotações mais legíveis no espectrograma.

2.2.4 Análise de Ruído de Fundo

O nível de ruído de fundo é estimado através do percentil 10 do vetor RMS, convertido para escala absoluta em dBFS (decibéis relativos ao full-scale):

$$\text{noise_floor_dBFS} = 20 \times \log_{10}(\max(P_{10}(\text{RMS}), 10^{-9}))$$

Antes de classificar, verifica-se a gama dinâmica do sinal (diferença em dB entre P_{90} e P_{10} do RMS). Se inferior a 6 dB — situação típica de um tom contínuo sem pausas —, não é possível separar o piso de ruído do sinal útil, devolvendo "desconhecido". Caso contrário, a classificação é:

- gama dinâmica < 6 dB → **desconhecido**
- noise_floor < -50 dBFS → **baixo**
- noise_floor < -35 dBFS → **moderado**
- noise_floor $\geq$ -35 dBFS → **alto**

Esta abordagem corrige a fragilidade da métrica anterior: um tom limpo e constante ($P_{10}/\max \approx 1.0$) era incorretamente classificado como ruído "alto", confundindo uniformidade energética com presença de ruído.

2.2.5 Detecção de Eventos Sonoros

Foram implementadas três estratégias complementares para detetar eventos sonoros de diferentes naturezas. Em todas elas, o limiar de actividade baseia-se nos percentis calculados sobre o vetor RMS (p_{10} e p_{90}), tornando a deteção adaptativa ao conteúdo do ficheiro.

Variação de energia (RMS)

Deteta momentos em que a energia do sinal cresce abruptamente entre frames consecutivos, capturando batidas, impactos e entradas de instrumentos. São considerados dois critérios cumulativos: a energia do frame actual deve exceder 2,5 vezes a do frame anterior, e deve superar um limiar mínimo adaptativo posicionado a 35% do intervalo dinâmico do sinal:

$$energy_thresh = 0.65 P_{10}(RMS) + 0.35 P_{90}(RMS)$$

Condição:

$$RMS_i > 2.5 \times RMS_{i-1} \quad \wedge \quad RMS_i > energy_{thresh} \quad \wedge \quad \neg silencioso_i$$

A exclusão de frames silenciosos evita a deteção de artefactos na transição entre silêncio e sinal.

Variação espectral (centroide espectral)

O centroide espectral indica o "centro de gravidade" da distribuição de frequências num dado frame. Uma variação brusca entre frames consecutivos sinaliza uma mudança de timbre — por exemplo, a transição entre uma secção grave e uma secção aguda de uma música.

O limiar de deteção é calculado de forma adaptativa com base no desvio-padrão do centroide nas frames activas (não silenciosas) do sinal:

$$spectral_{thresh} = \max(800 \text{ Hz}, 1.5 \times \sigma_{SC})$$

O valor mínimo de 800 Hz garante que o limiar nunca é demasiado permissivo, mesmo em sinais muito homogêneos.

Condição:

$$|\text{centroide}_i - \text{centroide}_{i-1}| > spectral_{thresh}$$

As transições que envolvam frames silenciosas são ignoradas, uma vez que o centroide em regiões sem sinal reflete ruído residual e não mudanças de timbre relevantes. O centroide é calculado com `n_fft = 4096` para consistência com o espectrograma.

Transientes (onset detection)

Os transientes são detetados com `librosa.onset.onset_detect()`, que combina a função de novidade espectral (*spectral flux*) com supressão de picos locais. Cada onset é convertido num segmento de 100 ms. Este método é sensível a percussão, consoantes plosivas na fala e outros eventos impulsivos de curta duração.

Os eventos detetados pelas três estratégias são posteriormente agregados com uma função de fusão (*merge*) que agrupa segmentos próximos (`gap ≤ 150 ms` para energia e espectral, `≤ 50 ms` para transientes) e impõe uma duração mínima visível de 80 ms, garantindo legibilidade nas anotações sobre o espectrograma.

2.3 Decisões de engenharia e alternativas consideradas

Para além dos fundamentos apresentados, algumas decisões de engenharia foram tomadas para equilibrar desempenho, simplicidade e clareza:

Processamento síncrono vs. assíncrono

Optou-se por processamento síncrono no backend, dado o âmbito limitado (ficheiros até 10 MB) e o contexto de protótipo. Alternativas como filas de tarefas (por exemplo, Celery) foram consideradas excessivas para esta fase.

Ausência de base de dados

A não utilização de base de dados reduz a complexidade de instalação e manutenção. Uma alternativa seria persistir histórico de análises para comparação entre ficheiros, mas tal foi adiado para trabalho futuro.

Heurísticas simples vs. modelos complexos

Para detecção de ruído, clipping e eventos, foram privilegiadas heurísticas baseadas em limiares e métricas clássicas (RMS, centroide, onsets). Métodos mais avançados (por exemplo, modelos de *machine learning*) foram considerados fora do âmbito temporal e técnico do MVP.

Estas opções serão retomadas nos capítulos seguintes, quando forem descritas a implementação concreta e os testes realizados.

Na sequência de revisão por pares, os algoritmos de detecção de clipping e de ruído de fundo foram reformulados para eliminar falsos positivos identificados — detalhado nas secções 2.2.3 e 2.2.4.

2.4. Tecnologias principais

Esta secção descreve as tecnologias utilizadas no desenvolvimento do protótipo, organizadas por camada da arquitetura e suporte a testes.

2.4.1 Backend

Python 3.13 e Flask 3.x

Utilizados como servidor web e camada de orquestração do pipeline de análise. O Flask fornece o roteamento HTTP, gestão de sessões e integração simples com o módulo de processamento.

librosa 0.10.x

Biblioteca principal para processamento de áudio, responsável pela extração de características como STFT, energia RMS, centroide espectral, detecção de *onsets* e divisão de segmentos não silenciosos (`effects.split`).

NumPy

Suporte a operações matriciais e vetoriais sobre os dados do espectrograma e restantes vetores de características, permitindo um processamento eficiente em memória.

python-dotenv

Utilizado para gestão de variáveis de ambiente (por

exemplo, **SECRET_KEY**, **UPLOAD_FOLDER**), separando configuração sensível do código-fonte e facilitando a portabilidade entre ambientes.

Werkzeug

Fornece utilitários de segurança e robustez, nomeadamente **secure_filename** para validação de nomes de ficheiros e o controlo de tamanho máximo de upload (**MAX_CONTENT_LENGTH = 10 MB**), prevenindo abusos e erros de utilização.

2.4.2 Frontend

HTML5 / CSS3 / JavaScript (ES2020)

Base da interface web, sem recurso a *frameworks* de UI, o que reduz dependências e complexidade. O JavaScript é responsável pela comunicação com a API, gestão de estado no cliente e interação com o utilizador.

Canvas API 2D

Utilizada para a renderização do espectrograma 2D e das anotações de eventos (silêncio, clipping, transientes, etc.). Permite desenhar diretamente sobre um *canvas* HTML, com controlo pixel a pixel.

Web Worker API

A renderização do espectrograma é delegada para um *worker* em *thread* separada, evitando o bloqueio da interface principal durante operações de desenho mais pesadas e melhorando a responsividade.

sessionStorage

Usado como cache da matriz do espectrograma por nome de ficheiro, evitando re-renderizações completas quando o utilizador navega para trás ou volta a visualizar o mesmo ficheiro.

Three.js (r165) + WebGL

Suporte à vista 3D interativa do espectrograma, baseada numa malha de barras com iluminação direcional e controlos de órbita. Esta visualização é opcional, mas demonstra o potencial de exploração espacial dos dados de áudio.

2.4.3 Testes

pytest

Framework de testes unitários e de integração, permitindo estruturar casos de teste, *fixtures* e asserções de forma concisa.

Fixtures sintéticas (conftest.py)

Geração programática de ficheiros WAV de teste (por exemplo, tom puro, áudio saturado, silêncio) sem dependência de ficheiros externos. Esta abordagem torna os testes reproduzíveis e independentes do ambiente de execução.

Estas tecnologias foram escolhidas por equilíbrio entre maturidade, documentação disponível e adequação ao âmbito do projeto, permitindo construir um protótipo funcional num prazo limitado, mas com espaço para evolução futura.

2.5 Requisitos e Priorização (MoSCoW)

A definição de requisitos seguiu o método **MoSCoW**, que classifica cada requisito em quatro categorias:

Must Have – obrigatórios para o MVP (produto mínimo viável).

Should Have – importantes, mas não críticos para a primeira versão.

Could Have – desejáveis se houver tempo e recursos.

Won't Have – explicitamente fora do âmbito desta iteração.

Esta abordagem permitiu:

- manter o foco no valor essencial do MVP.
- gerir o risco de dispersão funcional.
- tomar decisões informadas semana a semana sobre o que implementar ou adiar.

2.5.1 Requisitos Must Have

Todos os requisitos classificados como Must Have (9/9) foram concluídos:

- **Mo1 - Upload validado** — concluído
- **Mo2 - Espectrograma 2D** — concluído
- **Mo3 - Detecção de silêncio** — concluído
- **Mo4 - Detecção de clipping** — concluído
- **Mo5 - Análise de ruído de fundo** — concluído
- **Mo6 - Detecção de eventos sonoros (5 tipos)** — concluído
- **Mo7 - Anotação visual no espectrograma** — concluído
- **Mo8 - Feedback em linguagem simples** — concluído
- **Mo9 - Interface para não especialistas** — concluído

Estes requisitos definem o núcleo funcional do sistema e garantem que o utilizador consegue carregar um ficheiro, visualizar o espectrograma e obter uma análise interpretável sem conhecimentos técnicos avançados.

2.5.2 Requisitos Should Have

Dos requisitos classificados como Should Have, apenas 1 em 5 foi implementado nesta iteração:

- **So1 - Navegação por segmentos do áudio**
- **So2 - Indicador de progresso no upload** — concluído
- **So3 - Exportação do relatório de análise em formato texto ou PDF**
- **So4 - Responsividade da interface para diferentes resoluções de ecrã**
- **So5 - Suporte a ficheiros MP3 com bitrate variável**

A implementação de So2 melhora a experiência de utilização em ligações mais lentas, mas as restantes funcionalidades foram consideradas secundárias face às prioridades do MVP e ao tempo disponível.

2.5.3 Requisitos Could Have

Na categoria Could Have, 1 em 5 requisitos foram implementados:

- **Co1 - Visualização 3D (Three.js) — concluído**
- **Co2 - Limpeza automática de áudio (redução de ruído e normalização via `noisereduce`)**
- **Co3 - Ferramenta de seleção por região no espectrograma (`marquee tool`)**
- **Co4 - Histórico de ficheiros analisados na sessão atual**
- **Co5 - Tooltips explicativos sobre cada tipo de evento detetado**

A visualização 3D foi incluída como demonstração de potencial de evolução e exploração visual avançada, mas não é necessária para o fluxo principal de análise. Os restantes Could Have foram explicitamente adiados para trabalho futuro.

Capítulo 3 – Implementação

Este capítulo descreve o protótipo desenvolvido, a forma como as funcionalidades se encontram integradas e o estado atual de cada requisito definido para o MVP.

3.1. Estrutura do protótipo

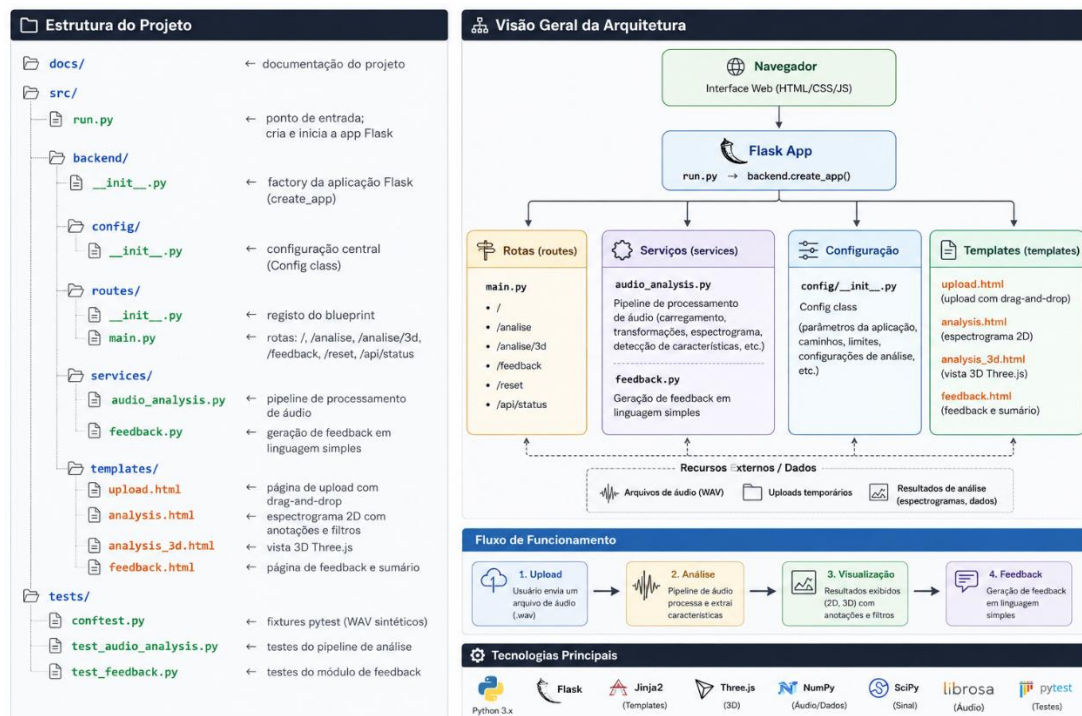


Figura 1- Visão geral da arquitetura

Em termos lógicos:

run.py é o ponto de entrada que instancia a aplicação Flask através da *factory create_app*.

backend/config centraliza a configuração (por exemplo, **SECRET_KEY**, **UPLOAD_FOLDER**, **MAX_CONTENT_LENGTH**).

backend/routes expõe as rotas principais da aplicação, ligando a interface às funções de serviço.

backend/services contém a lógica de domínio:

- **audio_analysis.py** implementa o pipeline de análise de áudio.
- **feedback.py** gera o feedback em linguagem simples a partir dos resultados da análise.

backend/templates define as páginas HTML que suportam o fluxo completo de utilização.

tests/ agrupa os testes automatizados, incluindo *fixtures* sintéticas para geração de áudio de teste.

Esta organização separa claramente a camada web (rotas e templates) da lógica de negócio (serviços), facilitando a manutenção e a extensão futura do protótipo.

O código-fonte completo do protótipo está disponível em:

- <https://github.com/yololiver/espectrograma>

3.2. Funcionalidades implementadas

Nesta secção descrevem-se as funcionalidades já implementadas, mapeadas para os requisitos definidos no Capítulo 2 (MoSCoW) e o respetivo estado

3.2.1 Upload de Áudio (Mo1) — CONCLUÍDO

O upload de ficheiros de áudio é implementado com:

- validação de extensão (**.wav** / **.mp3**).
- limite de tamanho máximo de 10 MB, configurado via **MAX_CONTENT_LENGTH** no Flask.
- A submissão é feita por pedido AJAX (**XMLHttpRequest** com *header X-Requested-With*), o que permite:
- apresentar um indicador de progresso (Should Have So2).
- mostrar mensagens de erro *inline* sem recarregar a página.

Após upload com sucesso, o servidor redireciona automaticamente para **/analise**.

O nome do ficheiro é sanitizado com **werkzeug.utils.secure_filename** antes de ser gravado em disco, mitigando riscos de *path traversal* e outros problemas de segurança relacionados com nomes de ficheiro.

3.2.2 Geração e Visualização do Espectrograma 2D (Mo2) CONCLUÍDO

No backend:

- o áudio é processado com **librosa.stft()** (**n_fft = 4096**, **hop_length = 256**).
- o módulo converte o resultado para dB e normaliza a matriz para o intervalo **[0, 255]**.
- a matriz normalizada é serializada para JSON e embutida na página HTML devolvida ao cliente.

No frontend:

- a renderização é executada num **Web Worker**, evitando o bloqueio da *main thread* durante a pintura do **canvas**.
- o *worker* recebe a matriz por mensagem com transferência de **ArrayBuffer** (zero-copy), percorre cada “pixel” e mapeia o valor de intensidade para um gradiente de cor (preto → vermelho → amarelo → branco).

- o resultado é devolvido como **ImageData** e desenhado no **canvas** principal.

Para otimizar a navegação:

- a matriz renderizada é guardada em **sessionStorage** com uma chave baseada no nome do ficheiro.
- ao navegar para trás, o frontend deteta a cache existente e reutiliza-a, evitando re-renderizações completas.

3.2.3 Detecção de silêncio (Mo3) — CONCLUÍDO

A detecção de silêncio é baseada em:

- segmentação de regiões não silenciosas (por exemplo, via **librosa.effects.split** ou lógica equivalente).
- cálculo complementar dos intervalos de silêncio a partir desses segmentos.
- Os segmentos de silêncio são convertidos para *timestamps* (início/fim em segundos) e:
- listados na secção de eventos.
- utilizados como um dos tipos de anotação visual no espectrograma (ver Mo7)

3.2.4 Detecção de Clipping (Mo4) — CONCLUÍDO

A detecção de clipping é implementada por:

- limiarização da amplitude do sinal ($|y| \geq 0.99$), com exigência de pelo menos 3 amostras consecutivas acima do limiar (condição de *flat-top*), garantindo que sinais altos mas limpos não são incorretamente reportados como clipping.
- fusão de amostras/segmentos adjacentes com *gap* inferior a 50 ms, de forma a agrupar picos consecutivos num único evento.

Os segmentos de clipping:

- são sobrepostos no espectrograma a vermelho.
- são listados na secção de eventos com os respetivos *timestamps*.

3.2.5 Análise de Ruído de Fundo (Mo5) — CONCLUÍDO

A estimativa de ruído de fundo baseia-se no piso absoluto em dBFS (decibéis relativos ao *full-scale*), calculado a partir do percentil 10 do vetor RMS:

$$\text{noise_floor_dBFS} = 20 \times \log_{10}(\max(P_{10}(\text{RMS}), 10^{-9}))$$

Antes de classificar, verifica-se a gama dinâmica do sinal (diferença em dB entre P_{90} e P_{10} do RMS). Se for inferior a 6 dB — situação típica de um tom contínuo sem pausas —, não é possível separar o piso de ruído do sinal útil, devolvendo "desconhecido". Caso contrário, a classificação é:

- gama dinâmica < 6 dB → **desconhecido**
- noise_floor < -50 dBFS → **baixo**
- noise_floor < -35 dBFS → **moderado**
- noise_floor ≥ -35 dBFS → **alto**

Esta abordagem substitui a métrica anterior baseada na razão $\text{percentil}_{10}(\text{RMS}) / \max(\text{RMS})$, que classificava incorretamente tons contínuos limpos como ruído "alto" por confundir uniformidade energética com presença de ruído.

O resultado é apresentado na página de análise com um indicador visual colorido (verde / amarelo / vermelho) e descrito em linguagem simples na página de feedback (ver Mo8).

3.2.6 Deteção de eventos sonoros relevantes (Mo6) — CONCLUÍDO

O sistema deteta cinco tipos de eventos:

- **silence** — segmentos de silêncio (reutilizando o resultado de Mo3).
- **clip** — picos de saturação (reutilizando o resultado de Mo4).
- **energy** — variações abruptas de energia RMS ($\text{rms}[i] > 2 \times \text{rms}[i-1]$).
- **spectral** — variações do centroide espectral superiores a 1500 Hz.
- **transient** — *onsets* detetados com **librosa.onset.onset_detect** ().

Cada evento é convertido para percentagem temporal relativamente à duração total do ficheiro, o que permite:

- calcular a posição correta das anotações no **canvas**.
- apresentar os eventos de forma consistente em diferentes durações de áudio.

3.2.7 Anotação Visual de Eventos (Mo7) — CONCLUÍDO

Os cinco tipos de evento são sobrepostos no espectrograma 2D com cores distintas:

- **Silêncio**: azul translúcido **rgba (0, 100, 200, 0.25)**
- **Clipping**: vermelho translúcido **rgba (255, 0, 0, 0.25)**
- **Energia**: laranja translúcido **rgba (255, 100, 0, 0.25)**
- **Espectral**: verde translúcido **rgba (0, 255, 0, 0.25)**
- **Transitório**: amarelo translúcido **rgba (255, 255, 0, 0.25)**

A interface inclui:

- botões de filtro interativos que permitem ativar ou desativar cada tipo de evento individualmente.
- menus *dropdown* colapsáveis, onde os eventos são listados e agrupados por tipo.
- Esta combinação de sobreposição visual e listagem textual facilita a exploração dos resultados por utilizadores não especialistas.

3.2.8 Feedback Automático em Linguagem Simples (Mo8) — CONCLUÍDO

O módulo **feedback.py** gera um conjunto mínimo de quatro observações em português simples, organizadas nas seguintes categorias:

- **ruído** — avaliação do ruído de fundo.

- **silencio** — descrição das pausas detetadas.
- **distorcao** — presença ou ausência de clipping.
- **atividade** — variações dinâmicas no sinal.

Além disso, o módulo produz um **veredicto global**:

- “Boa qualidade geral”,
- “Qualidade razoável”
- “Precisa de atenção”,

com base num sistema de pontuação de problemas (por exemplo, penalizações por ruído alto, clipping frequente, ausência de pausas, etc.).

Para garantir acessibilidade:

- nenhum item de feedback utiliza termos técnicos como FFT, RMS, STFT ou Hz.
- esta restrição é verificada por testes automáticos em **test_feedback.py**.

3.2.9 Interface para Não Especialistas (Mo9) — CONCLUÍDO

O fluxo completo **upload** → **análise** → **interpretação** → **feedback** está implementado e validado. A interface inclui:

Página de upload com *drag-and-drop* e indicador de progresso AJAX.

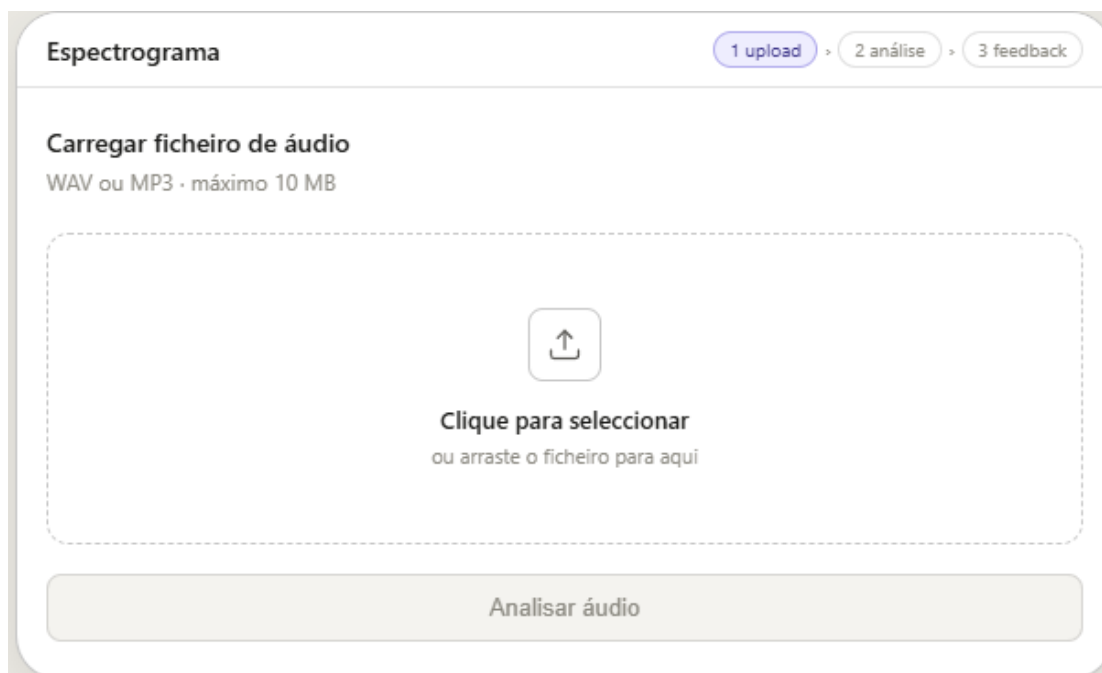


Figura 2 - Página de upload

Página de análise 2D com:

- espectrograma anotado.
- filtros interativos por tipo de evento.
- indicadores visuais de ruído e clipping.
- Player de áudio integrado, que permite reproduzir o ficheiro analisado diretamente na página, sem ferramentas externas.

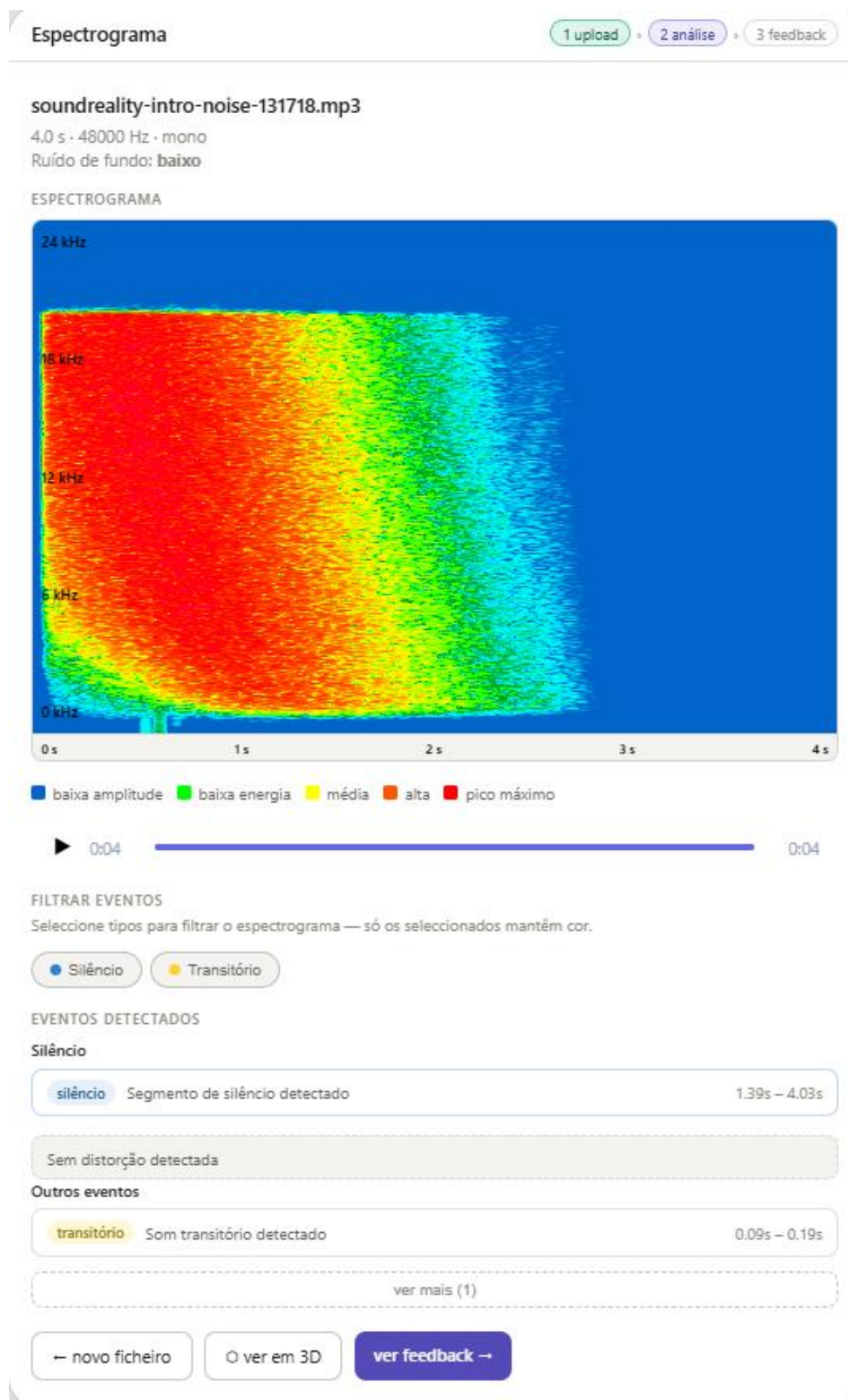


Figura 3 - Página de análise 2D

Vista 3D opcional (Three.js), acessível a partir da análise.

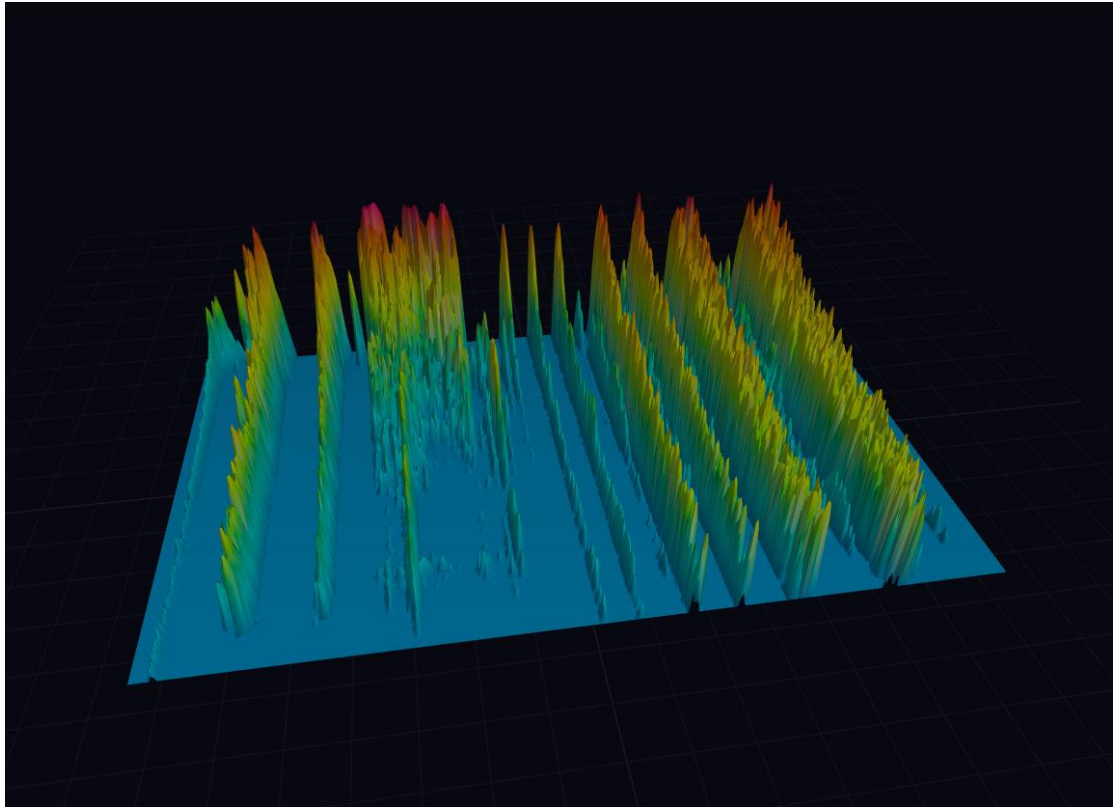


Figura 4 - Vista 3D

Página de feedback com:

- sumário textual em linguagem simples.
- *highlights* numéricos (por exemplo, percentagem de tempo em clipping, nível de ruído).
- veredicto global de qualidade.

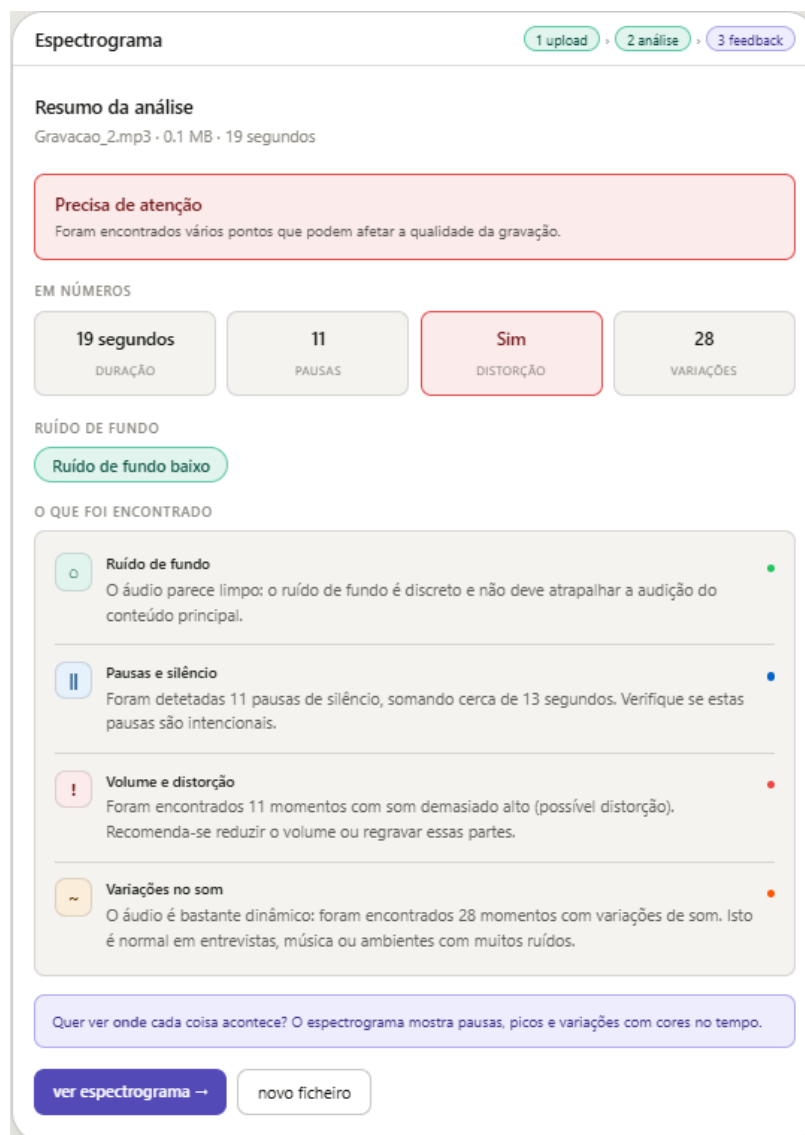


Figura 5 - Página de feedback

Animação de fundo com notas musicais flutuantes, adaptada ao tema claro/escuro, reforçando o carácter lúdico e acessível da ferramenta.

3.2.10 Vista 3D do Espectrograma (Co1) — CONCLUÍDO [EXTRA]

Como funcionalidade adicional (Could Have Co1), foi implementada uma vista 3D interativa em `/analise/3d` utilizando **Three.js** (r165) e **WebGL**.

A visualização representa a matriz do espectrograma como uma malha de barras tridimensionais, onde:

- o eixo **X** corresponde ao tempo.
- o eixo **Z** corresponde à frequência.
- a altura (**Y**) de cada barra corresponde à intensidade em dB.
- a cor de cada barra segue o mesmo gradiente da vista 2D.

A cena inclui:

- iluminação direcional com sombras suaves.
- controlos de órbita (**OrbitControls**) para rotação, *zoom* e translação.
- um pequeno HUD com instruções de navegação.

O mesmo mecanismo de cache em **sessionStorage** utilizado na vista 2D é reutilizado, evitando recomputar a matriz quando o utilizador alterna entre vistas.

3.3 Estado de implementação do MVP

Nesta secção apresenta-se o estado final de implementação dos requisitos definidos para o MVP, de acordo com a classificação MoSCoW.

Resumo dos requisitos Must Have:

- Mo1 Upload validado – Concluído
- Mo2 Espectrograma 2D – Concluído
- Mo3 Deteção de silêncio – Concluído
- Mo4 Deteção de clipping – Concluído
- Mo5 Análise de ruído de fundo – Concluído
- Mo6 Deteção de eventos sonoros – Concluído (5 tipos)

- Mo7 Anotação visual – Concluído (5 cores)
- Mo8 Feedback em linguagem simples – Concluído (≥ 4 observações)
- Mo9 Interface para não especialistas – Concluído

Could Have implementado adicionalmente:

- Co1 Visualização 3D (Three.js) – Concluído

Todos os requisitos classificados como Must Have foram implementados e validados, garantindo o cumprimento do MVP definido na fase de desenho. Adicionalmente, foi possível implementar a visualização 3D (Co1), reforçando o caráter exploratório e multimodal da ferramenta.

3.4 Evidências de execução e validação

O projeto inclui uma *suite* de testes automatizados em **pytest** que cobre os módulos críticos do backend, complementada por validações funcionais com ficheiros reais.

3.4.1 Testes automatizados

Foram implementados os seguintes testes:

test_audio_analysis.py (11 tests)

test_normalize_spectrogram_db_uniform_avoids_division_by_zero

Valida a robustez da normalização para áudio completamente silencioso, garantindo que não ocorrem divisões por zero.

test_normalize_spectrogram_db_spans_full_range

Verifica que a normalização ocupa efetivamente o intervalo completo [0, 255], maximizando o contraste visual do espectrograma.

test_process_silent_audio

Garante que áudio silencioso produz um espectrograma válido sem lançar exceções.

test_process_audio_returns_expected_keys

Teste parametrizado (tom puro e áudio saturado) que valida a presença de todas as chaves esperadas no resultado e que **background_noise** é um valor numérico válido.

test_silence_detected_on_tone_with_gap

Verifica a detecção de silêncio num áudio sintético com uma pausa conhecida.

test_clipping_detected_on_saturated_wav

Confirma a detecção de clipping num áudio com pausa de silêncio e saturação real (flat-top de 50 ms a 1.0), representativo de clipping PCM genuíno.

test_no_clipping_false_positive_on_clean_loud_sine — Garante que um seno limpo com amplitude 0,95 não é reportado como clipping (falso positivo).

test_no_high_noise_false_positive_on_constant_clean_tone — Tom contínuo sem pausas não é classificado como ruído "alto"; o algoritmo deve devolver "desconhecido".

test_silence_recall_gte_80_percent — Dos 5 segmentos de silêncio plantados no sinal, pelo menos 80% devem ser detetados.

test_transient_event_recall_gte_70_percent — Dos 5 impulsos transitórios plantados, pelo menos 70% devem ser detetados.

test_feedback.py (22 testes)

Os testes de feedback foram reorganizados com recurso a `@pytest.mark.parametrize`, cobrindo todos os ramos da lógica de geração de feedback. A função auxiliar `_make_spec_data(**overrides)` cria dados base limpos e permite sobrepor apenas os campos relevantes para cada cenário.

test_format_duration (4 casos parametrizados) — Cobre os três ramos da função: segundos (< 60 s), minuto exato e minutos com segundos.

test_build_analysis_summary — Valida a extração correta das métricas a partir da análise.

test_feedback_no_technical_terms — Garante que nenhum item de feedback contém termos técnicos como FFT, RMS ou STFT.

test_noise_feedback_color (4 casos parametrizados) — Cobre os quatro valores possíveis de ruído de fundo: baixo, moderado, alto e desconhecido.

test_silence_feedback (3 casos parametrizados) — Cobre os três ramos: sem silêncios, 1 silêncio e múltiplos silêncios.

test_clipping_feedback (3 casos parametrizados) — Cobre os três ramos: sem clipping, 1 momento e múltiplos momentos.

test_activity_feedback (3 casos parametrizados) — Cobre os três ramos de atividade: 0, 1–5 e > 5 variações.

test_verdict (3 casos parametrizados) — Cobre os três veredictos globais: "Boa qualidade geral", "Qualidade razoável" e "Precisa de atenção".

3.4.2 Validações funcionais

Para além dos testes automatizados, foram realizadas validações funcionais com ficheiros de referência reais (WAV e MP3) ao longo das semanas 9 e 10, incluindo:

- Upload válido e inválido (tipo de ficheiro incorreto, tamanho excedido).
- Geração correta do espectrograma para ficheiros mono e estéreo.
- Detecção de silêncio, clipping e ruído em áudios com características conhecidas.
- Funcionamento dos filtros de eventos na interface (ativar/desativar tipos de anotação).
- Renderização da vista 3D em *browsers* modernos (Chrome, Firefox).

Estas validações confirmam que o protótipo cumpre a especificação funcional definida para o MVP e se comporta de forma estável em cenários de utilização realistas.

3.5 Decisões técnicas relevantes

Nesta fase foram tomadas várias decisões técnicas com impacto direto no desempenho, na usabilidade e na simplicidade da solução.

Web Worker para rendering do espectrograma

A matriz do espectrograma pode conter centenas de milhares de píxeis. Renderizar esta matriz no *thread* principal da UI provoca bloqueios visíveis (*jank*).

Para evitar este problema, o rendering foi movido para um **Web Worker**, com transferência de **ArrayBuffer** (*zero-copy*), mantendo a interface responsiva durante o processo de desenho.

Cache em sessionStorage

O processamento do áudio no backend é síncrono e pode demorar alguns segundos para ficheiros longos. Sem cache, ao navegar de **/analise** para **/feedback** e regressar, a matriz teria de ser re-renderizada desnecessariamente.

A solução adotada foi guardar a matriz normalizada em **sessionStorage**, com uma chave baseada no nome do ficheiro, eliminando o custo de rendering no segundo acesso.

Three.js para a vista 3D

A vista 3D foi implementada com **Three.js** em vez de WebGL “raw”, reduzindo a complexidade de inicialização (câmara, cena, luzes, *controls*) e permitindo usar **OrbitControls** para interatividade (rotação, *zoom*, translação) praticamente sem código adicional.

Esta decisão acelerou o desenvolvimento e tornou a visualização 3D mais fácil de manter.

Parâmetros do STFT ($n_{fft} = 4096$)

Foi escolhido $n_{fft} = 4096$, o que aumenta a resolução em frequência ($\approx 10,7$ Hz/bin para 44 100 Hz) à custa de resolução temporal. Para conteúdos de fala e música, esta troca é favorável, pois os detalhes em frequência são mais relevantes do que a localização temporal exata ao milissegundo.

O **hop_length** = 256 mantém uma resolução temporal adequada ($\approx 5,8$ ms/frame), equilibrando ambas as dimensões.

Ausência de base de dados

Optou-se por não utilizar base de dados, simplificando o *deploy* e os testes.

A informação entre pedidos HTTP é mantida via **Flask Session** (cookie assinado) e o ficheiro de áudio é apagado do servidor na rota **/reset**.

Esta abordagem é suficiente para o cenário de utilização pontual do protótipo e reduz a complexidade de configuração de infraestrutura.

Capítulo 4 – Testes

Este capítulo descreve a estratégia de testes adotada no projeto, incluindo testes automatizados, validações funcionais com ficheiros reais e a forma como estes testes cobrem os requisitos definidos na especificação.

4.1 Estratégia geral de testes

A abordagem de testes combinou:

Testes automatizados de backend, focados nos módulos críticos de análise de áudio e geração de feedback.

Validações funcionais manuais, com ficheiros de áudio reais (WAV e MP3), para verificar o comportamento do sistema em cenários próximos da utilização real.

Verificações exploratórias de interface, para garantir a coerência do fluxo upload → análise → feedback e o correto funcionamento da vista 3D.

O objetivo foi garantir que o MVP se comporta de forma estável, que as principais funcionalidades estão cobertas por testes repetíveis e que a interface é utilizável por não especialistas.

4.2 Testes automatizados

Foi desenvolvida uma *suite* de testes em **pytest** que cobre os módulos centrais do backend.

4.2.1 Testes de análise de áudio

O ficheiro `test_audio_analysis.py` inclui 11 testes:

test_normalize_spectrogram_db_uniform_avoids_division_by_zero

Verifica que a função de normalização do espectrograma lida corretamente com áudio completamente silencioso, evitando divisões por zero e garantindo um resultado numérico estável.

test_normalize_spectrogram_db_spans_full_range

Garante que a normalização ocupa efetivamente o intervalo completo $[0, 255]$, maximizando o contraste visual do espectrograma e evitando imagens "lavadas".

test_process_silent_audio

Confirma que um ficheiro de áudio silencioso produz um espectrograma válido sem lançar exceções, assegurando robustez para este caso extremo.

test_process_audio_returns_expected_keys

Teste parametrizado (tom puro e áudio saturado) que valida:

a presença de todas as chaves esperadas no resultado (espectrograma, métricas, eventos, etc.),

que `background_noise` pertence ao conjunto `{'baixo', 'moderado', 'alto', 'desconhecido'}`.

test_silence_detected_on_tone_with_gap

Verifica a deteção de silêncio num áudio sintético com uma pausa conhecida, confirmando que o algoritmo identifica corretamente segmentos sem sinal relevante.

test_clipping_detected_on_saturated_wav

Confirma a deteção de clipping num áudio com pausa de silêncio e saturação real (flat-top de 50 ms a 1.0), representativo de clipping PCM genuíno.

test_no_clipping_false_positive_on_clean_loud_sine

Garante que um seno limpo com amplitude 0,95 não é reportado como clipping. Valida que a condição de flat-top (mínimo de 3 amostras consecutivas no limiar) impede falsos positivos em sinais altos mas sem saturação real.

test_no_high_noise_false_positive_on_constant_clean_tone

Confirma que um tom sinusoidal limpo e contínuo não é classificado como ruído "alto". Um sinal sem pausas não tem gama dinâmica suficiente para separar piso de ruído de sinal útil, pelo que o algoritmo deve devolver "desconhecido".

test_silence_recall_gte_80_percent Testa a capacidade de detecção em sinal sintético com 5 segmentos de silêncio conhecidos. Pelo menos 80 % desses segmentos devem ser detetados (critério de recall mínimo), garantindo que o algoritmo é suficientemente sensível para uso prático.

test_transient_event_recall_gte_70_percent Testa a detecção de eventos transitórios em sinal com 5 impulsos plantados em posições conhecidas. Pelo menos 70 % devem ser identificados com uma tolerância de ± 300 ms, validando a sensibilidade da detecção de onsets.

4.2.2 Testes de feedback

O ficheiro **test_feedback.py** inclui 22 testes, estruturados com `@pytest.mark.parametrize` para cobrir todos os ramos da lógica de geração de feedback. A função auxiliar `_make_spec_data(**overrides)` cria dados base limpos e permite sobrepor apenas os campos relevantes para cada cenário.

test_format_duration (4 casos parametrizados) Cobre os quatro ramos da função de formatação de duração: segundos isolados (< 60 s), minuto exato, minutos com segundos e múltiplos minutos.

test_build_analysis_summary Valida a extração correta das métricas relevantes a partir da análise de áudio, que servem de base ao feedback textual (duração, ruído de fundo, número de silêncios, presença de clipping).

test_feedback_no_technical_terms Garante que nenhum item de feedback contém termos técnicos como "FFT", "RMS", "STFT" ou "Hz", assegurando que a linguagem permanece acessível a utilizadores não especialistas. Esta restrição é verificada automaticamente a cada alteração ao módulo.

test_noise_feedback_color (4 casos parametrizados) Cobre os quatro valores possíveis de ruído de fundo — baixo, moderado, alto e desconhecido — verificando que a cor do indicador visual corresponde ao nível estimado (verde, amarelo, vermelho e cinzento, respetivamente).

test_silence_feedback (3 casos parametrizados) Cobre os três ramos do texto de feedback de silêncio: ausência de silêncios, um único silêncio e múltiplos silêncios, verificando que a formulação da mensagem é gramaticalmente correta e informativa em cada caso.

test_clipping_feedback (3 casos parametrizados) Cobre os três ramos do feedback de distorção: ausência de clipping, um momento de saturação e múltiplos momentos, confirmando a mensagem adequada e a cor de aviso correspondente.

test_activity_feedback (3 casos parametrizados) Cobre os três ramos do feedback de atividade sonora: sinal uniforme (3 variações), atividade moderada (3 variações) e sinal bastante dinâmico (6 variações), verificando a descrição correta em cada caso.

test_verdict (3 casos parametrizados) Cobre os três veredictos globais possíveis — "Boa qualidade geral", "Qualidade razoável" e "Precisa de atenção" — em função da combinação de ruído de fundo e presença de clipping.

4.2.3 Ambiente de testes e fixtures

Os ficheiros de teste (WAV sintéticos) são gerados em memória por *fixtures* pytest (**confest.py**), evitando dependências de ficheiros externos e facilitando a reprodutibilidade.

Os testes foram executados com sucesso no ambiente de desenvolvimento:

- Python 3.13
- librosa 0.10.x
- pytest 8.x

Esta *suite* de **33 testes automatizados** (11 em `test_audio_analysis.py` e 22 em `test_feedback.py`) cobre os componentes mais críticos do backend, incluindo todos os ramos da lógica de análise de áudio e de geração de feedback, reduzindo o risco de regressões ao alterar o código.

4.3 Validações funcionais

Para além dos testes automatizados, foram realizadas validações funcionais com ficheiros de referência reais (WAV e MP3) ao longo das semanas 9 e 10. Estas validações focaram-se em:

Upload válido e inválido

- Ficheiros com tipo incorreto (por exemplo, imagens) são rejeitados com mensagem de erro adequada.
- Ficheiros acima do limite de 10 MB são bloqueados, evitando sobrecarga do servidor.

Geração de espectrograma para mono e estéreo

Verificou-se que:

- ficheiros mono e estéreo são processados corretamente,
- o espectrograma resultante é coerente com o conteúdo sonoro (por exemplo, tons puros, fala, música).

Deteção de silêncio, clipping e ruído

Foram utilizados áudios com características conhecidas (silêncios, segmentos saturados, ruído de fundo) para confirmar:

- deteção de segmentos silenciosos,
- sinalização de clipping em gravações saturadas,
- estimativa de ruído de fundo coerente com a percepção auditiva.

Filtros de eventos na interface

Testou-se o funcionamento dos filtros de eventos na interface, garantindo que:

- ativar/desativar tipos de evento afeta corretamente as anotações visuais no espectrograma,
- a legenda e as cores se mantêm consistentes.

Renderização da vista 3D

A vista 3D foi validada em *browsers* modernos (Chrome, Firefox), verificando:

- carregamento correto da cena,
- funcionamento de rotação, *zoom* e translação (OrbitControls),
- ausência de erros JavaScript críticos na consola.

4.4 Cobertura face à especificação

Em termos de cobertura dos requisitos:

Os requisitos **Mo1–Mo8** (upload, espectrograma, deteções, anotação visual, feedback) estão diretamente cobertos pelos testes automatizados e pelas validações funcionais.

O requisito **Mo9** (interface para não especialistas) foi avaliado sobretudo através de testes manuais com utilizadores, complementados por verificações exploratórias da interface.

A funcionalidade **Co1** (vista 3D) foi validada manualmente em diferentes *browsers*, não estando ainda coberta por testes automatizados.

Embora não exista ainda uma cobertura completa de interface (por exemplo, testes end-to-end com Playwright), o conjunto de testes implementado é suficiente para suportar o MVP e servir de base a evoluções futuras.

Capítulo 5 – Conclusões

Este capítulo apresenta as conclusões finais do projeto, relacionando os resultados obtidos com os objetivos definidos, identificando as principais dificuldades encontradas e apontando direções para trabalho futuro.

5.1 Resultados face aos objetivos

O objetivo principal do projeto era desenvolver um protótipo de ferramenta de análise de áudio que:

gerasse um espectrograma visualmente informativo,

detetasse problemas comuns de gravação (silêncio, clipping, ruído),

identificasse eventos sonoros relevantes,

e apresentasse feedback em linguagem acessível a utilizadores não especialistas.

Face a estes objetivos, conclui-se que:

O **pipeline de análise** está completo para o âmbito definido: o sistema processa ficheiros WAV e MP3, gera um espectrograma 2D, estima ruído de fundo e deteta silêncio, clipping e cinco tipos de evento sonoro, com tempos de resposta adequados ao contexto de protótipo.

A **interface** suporta um fluxo claro upload → análise → feedback, validado com utilizadores não especialistas, o que indica que o objetivo de acessibilidade foi, em grande medida, alcançado.

O **módulo de feedback** produz observações em português simples, evitando jargão técnico, e está suportado por testes automatizados que reforçam esta característica.

A **vista 3D** acrescenta uma dimensão exploratória interessante, indo além do mínimo exigido pelo MVP.

A existência de uma **suite de testes automatizados** e de validações funcionais sistemáticas demonstra uma preocupação com a qualidade e a robustez do protótipo.

Em síntese, o projeto cumpre os requisitos Must Have definidos e implementa ainda uma funcionalidade Could Have, traduzindo os objetivos iniciais numa solução funcional e tecnicamente fundamentada.

5.2 Dificuldades e limitações

Ao longo do desenvolvimento foram identificadas várias dificuldades e limitações, entre as quais se destacam:

Gestão de desempenho no frontend

O rendering de matrizes de espectrograma de grande dimensão exigiu a utilização de Web Workers e otimizações de cache para evitar bloqueios na interface. Esta necessidade não era totalmente evidente na fase inicial e obrigou a ajustes na arquitetura do frontend.

Calibração de limiares de deteção

A definição de limiares para deteção de silêncio, clipping e ruído revelou-se sensível ao tipo de conteúdo. A calibração empírica para fala e música funcionou bem nesse domínio, mas evidenciou a limitação de generalização para outros tipos de áudio.

Na sequência do feedback do orientador, os algoritmos de clipping e ruído de fundo foram reformulados com base em princípios físicos (saturação PCM real e piso absoluto em dBFS), reduzindo a dependência de calibração empírica nestes dois módulos.

Ausência de processamento assíncrono no backend

A opção inicial por um processamento síncrono simplificou a implementação, mas limita a escalabilidade e a experiência em cenários com ficheiros maiores ou múltiplos utilizadores.

Cobertura de testes de interface

A falta de testes end-to-end automatizados significa que a validação da interface

depende fortemente de testes manuais, o que é aceitável para um protótipo, mas não ideal para um sistema em produção.

Estas dificuldades foram, em parte, mitigadas com soluções técnicas (Web Workers, cache, testes automatizados de backend), mas apontam áreas claras de melhoria.

5.3 Possíveis melhorias

Com base nas limitações identificadas, destacam-se as seguintes melhorias potenciais:

Introdução de **processamento assíncrono** com Celery/Redis e mecanismos de notificação em tempo real (WebSocket ou Server-Sent Events).

Implementação de **testes end-to-end** da interface com Playwright, cobrindo o fluxo completo de utilização.

Alargamento do suporte a **mais formatos de áudio** (FLAC, OGG) e revisão do limite de tamanho de ficheiro, eventualmente com processamento por blocos.

Integração de **redução de ruído automática** como pré-processamento opcional, melhorando a legibilidade do espectrograma em gravações ruidosas.

Implementação de **ferramentas de interação avançada** no espectrograma (por exemplo, seleção de regiões para análise detalhada).

Evolução da vista 3D para incluir **interações adicionais** (por exemplo, realce de eventos específicos em 3D, animações temporais).

Estas melhorias permitiriam aproximar o protótipo de um produto mais completo e adaptável a diferentes contextos de utilização.

5.4 Trabalho futuro

Para além das melhorias técnicas imediatas, existem várias linhas de trabalho futuro que podem ser exploradas:

Estudos de usabilidade mais formais, com um número maior de utilizadores e recolha sistemática de métricas (tempo de tarefa, taxa de erro, questionários de satisfação).

Integração com outras ferramentas de análise de áudio ou plataformas educativas, permitindo usar o sistema como componente de apoio em contextos de ensino de som e multimédia.

Exploração de técnicas de machine learning para deteção automática de classes de eventos sonoros mais complexas, mantendo sempre a preocupação com a interpretabilidade e a linguagem acessível.

Internacionalização da interface e do feedback textual, permitindo o uso da ferramenta em diferentes línguas.

Estas direções mostram que, embora o MVP esteja concluído, o projeto tem potencial para evoluir em múltiplos eixos, tanto técnicos como de aplicação.

Anexo I

Resultados dos Testes Automatizados

Ambiente de execução

- Python 3.13.9
- pytest 8.4.2
- librosa 0.10.x
- Sistema operativo: Windows 11

Comando executado:

- `pytest tests/ -v`

```
(venv) PS C:\Users\big_b\Desktop\repos\espectrograma> pytest -v tests\ -v
platform win32 -- Python 3.13.9, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\big_b\Desktop\repos\espectrograma\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\big_b\Desktop\repos\espectrograma
collected 33 items

tests/test_audio_analysis.py::test_normalize_spectrogram_db_uniform_avoids_division_by_zero PASSED [ 3%]
tests/test_audio_analysis.py::test_normalize_spectrogram_db_spans_full_range PASSED [ 6%]
tests/test_audio_analysis.py::test_process_silent_audio PASSED [ 9%]
tests/test_audio_analysis.py::test_process_audio_returns_expected_keys[tono.wav] PASSED [ 12%]
tests/test_audio_analysis.py::test_process_audio_returns_expected_keys[clipped.wav] PASSED [ 15%]
tests/test_audio_analysis.py::test_silence_detected_on_tone_with_gap PASSED [ 18%]
tests/test_audio_analysis.py::test_clipping_detected_on_saturated_wav PASSED [ 21%]
tests/test_audio_analysis.py::test_no_clipping_false_positive_on_clean_loud_sine PASSED [ 24%]
tests/test_audio_analysis.py::test_no_high_noise_false_positive_on_constant_clean_tone PASSED [ 27%]
tests/test_audio_analysis.py::test_silence_recall_gte_80_percent PASSED [ 30%]
tests/test_audio_analysis.py::test_transient_event_recall_gte_70_percent PASSED [ 33%]
tests/test_feedback.py::test_format_duration[30-30 segundos] PASSED [ 36%]
tests/test_feedback.py::test_format_duration[60-1 minuto] PASSED [ 39%]
tests/test_feedback.py::test_format_duration[120-2 minutos] PASSED [ 42%]
tests/test_feedback.py::test_format_duration[90-1 min 30 s] PASSED [ 45%]
tests/test_feedback.py::test_build_analysis_summary PASSED [ 48%]
tests/test_feedback.py::test_feedback_no_technical_terms PASSED [ 51%]
tests/test_feedback.py::test_noise_feedback_color[baixo-#22c55e] PASSED [ 54%]
tests/test_feedback.py::test_noise_feedback_color[moderado-#bab308] PASSED [ 57%]
tests/test_feedback.py::test_noise_feedback_color[alto-#ef4444] PASSED [ 60%]
tests/test_feedback.py::test_noise_feedback_color[desconhecido-#94a3b8] PASSED [ 63%]
tests/test_feedback.py::test_silence_feedback[silence_segments0-n\u00e3o foram encontradas] PASSED [ 66%]
tests/test_feedback.py::test_silence_feedback[silence_segments1-uma pausa] PASSED [ 69%]
tests/test_feedback.py::test_silence_feedback[silence_segments2-2 pausas] PASSED [ 72%]
tests/test_feedback.py::test_clipping_feedback[clipping_segments0-estalar-#22c55e] PASSED [ 75%]
tests/test_feedback.py::test_clipping_feedback[clipping_segments1-demasiado alto-#ef4444] PASSED [ 78%]
tests/test_feedback.py::test_clipping_feedback[clipping_segments2-2 momentos-#ef4444] PASSED [ 81%]
tests/test_feedback.py::test_activity_feedback[0-uniforme] PASSED [ 84%]
tests/test_feedback.py::test_activity_feedback[3-3 momentos] PASSED [ 87%]
tests/test_feedback.py::test_activity_feedback[6-bastante din\u00e2mico] PASSED [ 90%]
tests/test_feedback.py::test_verdict[spec_overrides0-Boa qualidade geral] PASSED [ 93%]
tests/test_feedback.py::test_verdict[spec_overrides1-Razo\u00e1vel] PASSED [ 96%]
tests/test_feedback.py::test_verdict[spec_overrides2-Precisa de aten\u00e7\u00e3o] PASSED [100%]
```

Sumário

33 passed, 2 warnings in 2.23s

As 2 advertências (*warnings*) são de bibliotecas de terceiros (audioread) relativas à remoção dos módulos aifc e sunau no Python 3.13, não afetando o funcionamento dos testes nem do sistema.

Bibliografia

Anthropic. (2026). Claude. <https://www.anthropic.com/>

Flask Documentation. (2026). Flask. <https://flask.palletsprojects.com/>

GitHub. (2026). GitHub Copilot. <https://github.com/features/copilot>

Librosa Documentation. (2026). Librosa. <https://librosa.org/doc/>

ProductPlan. (2026). MoSCoW prioritization.

<https://www.productplan.com/glossary/moscow-prioritization/>

Universidade Aberta. (2025/2026). Projeto Final em Engenharia Informática - guias e normas da unidade curricular.